# Pacman AI — Implementation Report

## 1. Implementation summary

The implementation is contained in `student_assignment.py`. The game engine in `game.py` is unchanged.

The shared evaluation function rewards survival, ghosts eliminated, ammunition, progress toward the next ammunition pickup, nearby shooting opportunities, and local mobility. It penalizes death heavily and considers ghost behavior when estimating danger. Manhattan distance is used for fast threat estimates; a small breadth-first search estimates nearby free space.

## 2. Minimax and alpha-beta

The player is the MAX side. The ghost response is the MIN side. The supplied engine advances ghosts automatically inside `PacmanGame.step()` and exposes no separate ghost-action API. To retain an explicit adversarial MIN layer without duplicating the entire engine transition, the search evaluates legal one-step responses for the two closest living ghosts while the remaining ghosts follow their built-in behavior. Ghost responses are generated on cloned states, resolved simultaneously, and followed by the engine's collision and ammunition rules.

This is a bounded adversarial approximation, not an exhaustive enumeration of every joint move of every ghost. Enumerating all joint ghost moves would grow combinatorially and risks exceeding the five-second move budget.

Minimax uses depth 2. Alpha-beta uses iterative deepening up to depth 3, orders promising player actions first, and orders the opponent responses from low to high evaluation. The internal deadline leaves a margin below the outer five-second process timeout. At equal completed depth and deterministic ghost-response sampling, minimax and alpha-beta use the same value model.

## 3. MCTS

MCTS uses UCT selection with the UCB1 score, expands one untried legal Pacman action, performs a guided stochastic rollout, and backpropagates the normalized evaluation value. The rollout prioritizes a shot when a ghost occupies an adjacent shooting tile. Otherwise, it chooses among movement actions biased toward approaching ghosts when ammunition is available and keeping distance when it is not.

The default budget is 180 iterations with a rollout depth of 10. The loop checks the time budget and stops before the five-second external limit. Ghost movement during MCTS uses the actual game engine dynamics.

## 4. Evaluation function

The evaluation function is from the player's perspective:

- A dead player receives a very large negative value.
- Eliminated ghosts receive a substantial positive reward.
- Remaining ammunition and progress toward the next ammunition pickup are rewarded.
- When ammunition is available, nearby ghosts and adjacent shooting opportunities are valuable.
- Without ammunition, distance from ghosts is rewarded and immediate danger is strongly penalized.
- Free neighboring tiles and reachable nearby space reward mobility.
- LOOKAHEAD and FOLLOW ghosts receive higher danger weights than RANDOM ghosts.

The scale is chosen so survival dominates small positional preferences and ghost elimination remains the principal objective.

## 5. Time-limit handling and validation

The existing wrapper enforces a five-second process limit and returns `Action.STAY` if an agent exceeds it. Minimax and alpha-beta also use internal deadlines; alpha-beta retains the action from the last fully completed iterative-deepening depth. MCTS checks its deadline during iteration and rollout.

The implementation was syntax-compiled, each agent was called on a live headless game state, returned actions were checked against `get_legal_actions()`, and short episodes were run for all three agents. A same-depth Minimax/alpha-beta decision check also returned the same action on the tested deterministic state.

The runtime used for these checks did not have Pygame installed, so the headless engine checks used a temporary import stub. The actual graphical window was not tested in this environment. Install the dependency from `requirements.txt` before running the project locally.